DDM501 · AI IN DEVOPS, DATAOPS, MLOPS

ML System Design Document

Real-time Transaction Fraud Detection for a Mobile E-wallet

Assignment	Individual Assignment 1 (weight 5%)
Student name	[Student Name]
Student ID	[Student ID]
Submission date	October 2026

Abstract. This document designs a production machine-learning system that scores every payment of a Vietnamese mobile e-wallet for fraud risk in real time. Today a hand-written rule engine stops less than half of the fraud value while blocking or delaying many legitimate payments. The proposed system adds a gradient-boosted risk model, streaming features and a cost-aware decision policy to the existing payment flow, keeps the rule engine as a safety net, and closes the loop with analyst and chargeback labels. The document covers the problem definition, requirements, a goal and metric hierarchy, the high-level architecture and the main design trade-offs.

The company, its numbers and its data are hypothetical but sized to be realistic for a mid-sized e-wallet in Vietnam.

Table of Contents

1. Problem Definition
   1.1 Context and Background
   1.2 Problem Statement
   1.3 Current Situation
   1.4 Why Machine Learning
   1.5 Stakeholders

2. Requirements Analysis
   2.1 Functional Requirements
   2.2 Non-Functional Requirements
   2.3 Data Requirements

3. Goals and Metrics
   3.1 Goal Hierarchy
   3.2 Business Goals
   3.3 System Goals
   3.4 Model Goals
   3.5 Baselines and Acceptance Thresholds

4. High-level Architecture Design
   4.1 Architecture Overview
   4.2 Data Flow
   4.3 ML Pipeline Stages
   4.4 Component Description

5. Trade-off Analysis
6. Risks and Rollout Plan
7. Conclusion

Glossary

Term	Meaning in this document
GMV	Gross merchandise value: total money moved through the wallet in a period.
Fraud loss rate	Fraud value that the company finally pays for, divided by GMV; quoted in basis points (1 bp = 0.01%).
False decline	A legitimate payment that is blocked or abandoned because of a fraud check.
Step-up / challenge	An extra authentication step (OTP, face match, PIN) required before a payment completes.
Chargeback / dispute	A customer or bank claim that a payment was unauthorised; the main source of confirmed fraud labels.
ATO	Account takeover: a fraudster gains control of a genuine customer's account.
PIT join	Point-in-time join: building training rows only from data that existed at the moment of the payment.
PR-AUC	Area under the precision–recall curve; preferred over ROC-AUC for very rare positive classes.

1. Problem Definition

1.1 Context and Background

WalletCo (a hypothetical company) runs a mobile e-wallet in Vietnam with about **6 million monthly active users** and **45,000 merchants**. Customers top up from linked bank accounts or cards, pay merchants by QR code, transfer money peer-to-peer (P2P), pay bills and buy phone credit. The platform processes about **1.2 million transactions per day** with an average value of 350,000 VND, i.e. roughly **12.6 trillion VND of GMV per month**. Traffic is very uneven: lunchtime, payday, the Tet holiday and "double-day" sales (9.9, 10.10, 11.11) produce peaks of 10–20× the daily average.

Because wallet payments are instant and irreversible, a fraudulent payment cannot be "pulled back" once it is settled. The main fraud patterns are:

- **Account takeover (ATO)** – stolen OTPs or SIM-swap, followed by fast transfers to mule accounts;
- **Stolen card / bank-link top-up** – a fraudster links a stolen card, tops up and cashes out;
- **Social-engineering (scam) transfers** – the real customer is tricked into sending money;
- **Promotion abuse** – farms of fake accounts collect cash-back vouchers;
- **Merchant collusion** – fake QR "sales" used to cash out stolen balances.

Fraud matters for three reasons: (1) direct financial loss, because WalletCo refunds victims; (2) regulatory exposure, since payment intermediaries licensed by the State Bank of Vietnam must manage fraud and protect customer data; and (3) trust – a customer who loses money or is wrongly blocked often leaves the platform.

1.2 Problem Statement

For every payment request, WalletCo must decide **within the payment's time budget (≤ 300 ms end-to-end, of which at most 100 ms is available for fraud checks)** whether to approve it, ask for step-up authentication, hold it for manual review, or block it. The current decision process has the following measured problems (last 6 months):

Problem	Current value	Impact
Fraud loss rate	≈ 6 bps of GMV (≈ 7.5 bn VND / month)	Direct loss refunded to customers; rising ~20% per quarter.
Share of fraud value stopped before settlement	≈ 45%	More than half of the fraud is discovered only after money has left.
False decline rate (good payments blocked or abandoned at challenge)	≈ 1.2% of transactions	≈ 14,000 good payments disrupted per day; complaints and churn.
Manual review queue	≈ 9,000 cases / day, precision ≈ 4%	25 analysts cannot keep up; median hold time 3.5 h.
Time to react to a new fraud pattern	2–4 weeks	New rules are written and tested by hand.

The **ML problem** is a **binary classification** task: estimate the probability that a payment is fraudulent, *P(fraud | payment, customer, device, merchant, history)*, and turn that probability into one of four actions

with a cost-aware policy. The target is to **halve the fraud loss rate while reducing false declines and analyst workload** (exact targets in Section 3).

1.3 Current Situation

Fraud is handled today by a **rule engine** of about 180 hand-written rules (for example, "block if more than 5 transfers in 10 minutes to new recipients" or "review if top-up > 20 M VND from a card linked < 24 h ago"), plus a fixed daily limit per account. The rules are evaluated synchronously inside the Payment Service. Payments that hit a "review" rule are held and sent to a queue that 25 fraud analysts work through in a spreadsheet-like back-office tool. Confirmed fraud is learned weeks later through customer complaints and bank chargebacks.

The approach has reached its limits:

- Rules look at one or two signals at a time and use fixed cut-offs, so they cannot combine weak signals (new device *and* unusual hour *and* new recipient *and* amount near the limit).
- Fraudsters learn the cut-offs and stay just below them.
- Every new rule adds false positives; nobody dares to delete old rules, so the false-decline rate keeps rising.
- There is no consistent record of *why* a payment was stopped, which makes customer-support and audit difficult.

1.4 Why Machine Learning is the Appropriate Approach

- **Pattern complexity:** fraud signals are weak individually and strong in combination; gradient-boosted trees learn these non-linear interactions from hundreds of features automatically.
- **Labelled history exists:** chargebacks, disputes and analyst verdicts give ≈ 350,000 confirmed fraud labels per year among ≈ 440 million transactions – enough for supervised learning despite the 0.08% positive rate.
- **Changing adversary:** fraud patterns drift monthly; a retrained model adapts in days instead of weeks of manual rule writing.
- **Graded output:** a calibrated probability allows a *graduated* response (allow / challenge / review / block) and a cost-based threshold, which binary rules cannot provide.
- **Proven practice:** ML risk scoring is standard in card networks and large payment companies, and the problem needs one model only, which keeps the system maintainable.

ML does not replace rules entirely. Regulatory and contractual constraints (sanction lists, daily limits, blocked devices) remain as **hard rules** evaluated before the model, and the rule engine is kept as the fallback when the model is unavailable.

1.5 Stakeholder Identification

Stakeholder	Goals	Concerns
Customers (end users)	Pay quickly; never lose money to fraud.	Being blocked or challenged on a normal payment; slow payments; how their personal data is used.
Merchants	High payment success rate at checkout.	Lost sales from false declines; held settlements.

Head of Payments (business owner)	Lower fraud loss, higher conversion and GMV, lower operating cost.	Return on investment; impact on customer experience; measurable results within 6 months.
Risk & Fraud Operations (analysts)	Catch fraud early; small, high-quality review queue.	Understanding *why* a payment was flagged; ability to override and to add urgent rules.
Compliance & Legal	Meet State Bank of Vietnam requirements and personal-data law (Decree 13/2023/ND-CP).	Explainable decisions, audit trail, data minimisation, no discriminatory features.
Data Science / ML team	Accurate, maintainable model; fast experimentation.	Label delay and noise, class imbalance, training–serving skew, reproducibility.
Platform / SRE team	Stable payment path.	Latency budget, availability, on-call load, safe rollback, infrastructure cost.
Customer Support	Resolve "why was I blocked?" tickets quickly.	Human-readable reason codes and a clear unblock process.

The most important tension is between **Risk Operations / Compliance** (stop more fraud, explain every decision) and **Customers / Merchants / Head of Payments** (fewer interruptions, more completed payments). The design resolves it with a calibrated score and an explicit cost model rather than with a single fixed cut-off.

2. Requirements Analysis

2.1 Functional Requirements

Core ML functionality

ID	Requirement
FR-1	Score every outgoing payment (QR payment, P2P transfer, bill payment, top-up, withdrawal) with a fraud probability in [0, 1] before authorisation.
FR-2	Map the score, amount and context to one of four actions: **ALLOW**, **CHALLENGE** (step-up OTP / face match), **REVIEW** (hold up to 30 min for an analyst) or **BLOCK**.
FR-3	Return the top 3 reason codes for every non-ALLOW decision (e.g. `NEW_DEVICE`, `VELOCITY_P2P_10M`, `RECIPIENT_MULE_RISK`), derived from feature attributions.
FR-4	Apply hard rules (sanction list, blocked device, regulatory limits) before the model; hard rules always win.
FR-5	Log every request, feature vector, score, decision, model version and latency for audit, monitoring and training.
FR-6	Ingest labels (analyst verdicts, disputes, chargebacks) and retrain the model on a schedule and on demand.
FR-7	Support shadow mode, canary release and one-click rollback of model versions.

Input / Output specification

Input (from Payment Service, JSON over gRPC/REST)	Output (to Payment Service)

`transaction_id`, `timestamp`, `type` (QR, P2P, BILL, TOPUP, WITHDRAW), `amount`, `currency`, `user_id` (pseudonymised), `recipient_id` or `merchant_id`, `funding_source` (balance / bank-link / card token), `device_id`, `ip`, `app_version`, `geo_cell` (coarse location), `session_id`. Server-side enrichment (not sent by caller): ≈ 120 features from the online feature store – velocity counts, recipient and merchant risk, device age, account age, KYC level, behavioural baselines.	`transaction_id`, `risk_score` (calibrated probability, 0–1), `decision` ∈ {ALLOW, CHALLENGE, REVIEW, BLOCK}, `reason_codes` (≤ 3), `model_version`, `policy_version`, `fallback_used` (bool), `latency_ms`.

Integration requirements

- **Payment Service:** synchronous call with a hard timeout of 150 ms; the Payment Service owns the final authorisation.
- **Event bus (Kafka):** the Payment Service, login service and device SDK publish events; the fraud system consumes them for streaming features and publishes its decisions.
- **Authentication Service:** executes CHALLENGE actions (OTP, face match) and reports the outcome back as a signal.
- **Case Management tool:** receives REVIEW cases with score and reasons; returns analyst verdicts as labels.
- **Data warehouse / data lake:** source of historical data and destination of decision logs; dispute and chargeback files from partner banks arrive daily.
- **Customer Support CRM:** read-only view of decisions and reason codes for a given transaction.

User interaction requirements

- **Customers** see a short, non-revealing message (“For your safety, please confirm with OTP”) and never the score.
- **Fraud analysts** see a case page with score, reason codes, recent account activity and linked accounts; they can approve, reject, or mark the account; each verdict is stored as a label.
- **Risk managers** can change decision thresholds and add emergency rules through a configuration UI, with four-eyes approval and versioning – no code deployment needed.
- **ML engineers** use a dashboard to compare model versions and promote or roll back a model.

2.2 Non-Functional Requirements

Category	Requirement
Performance	Scoring API latency (including feature lookup and model inference): **p50 ≤ 30 ms, p99 ≤ 100 ms**. Streaming features visible to the scorer within **≤ 5 s** of the event (p95). Analyst case page loads in ≤ 2 s.
Throughput & scalability	Average 15 TPS, normal daily peak 150 TPS, event peak (Tet, 11.11) **300 TPS**; design target **1,000 TPS** to cover 3 years of growth. Stateless scoring pods on Kubernetes with horizontal autoscaling on CPU and request rate; Redis Cluster for online features; Kafka topics partitioned by `user_id`. Pre-scale before known events.

Reliability	Availability **≥ 99.95%** per month (≤ 22 min downtime), deployed across 2 availability zones. **Graceful degradation:** if the scorer times out or errors, the Payment Service falls back to the existing rule engine; if only some features are missing, the model scores with default values and the decision is marked `degraded`. Fail-safe policy: low-value payments fail open (rules only); payments above 5 M VND are challenged. Circuit breaker opens after 5% errors in 30 s. No single point of failure.
Maintainability	Automatic **weekly retraining** on a rolling 12-month window; emergency retrain within 24 h when drift alerts fire. All models versioned with data snapshot, code commit and metrics; rollback to the previous version in **≤ 5 min**. Feature definitions kept as code and shared by training and serving. Monitoring of latency, error rate, score distribution, feature drift (PSI), decision mix, and delayed precision/recall once labels mature.
Security & compliance	mTLS between services; encryption at rest; role-based access to raw data; full audit log of decisions, threshold changes and model promotions kept for 5 years; explanations available for every non-ALLOW decision.

2.3 Data Requirements

Data sources

Source	Content	Arrival / use
Transaction events	All payment attempts with amount, type, parties, funding source, status.	Real-time (Kafka) for features; daily snapshot to the lake for training.
Account & KYC profile	Account age, KYC level, linked banks/cards (tokenised), limits.	Change-data-capture from the core database.
Device & session	Device fingerprint, OS, app version, IP, coarse location, login and OTP events.	Real-time from the mobile SDK and login service.
Merchant data	Category (MCC), age, historical refund and dispute rates.	Daily batch.
Labels	Analyst verdicts; customer fraud reports and disputes; bank chargebacks; confirmed mule accounts.	Verdicts within hours; disputes within days; chargebacks up to **60 days** later.
External (optional)	Telco SIM-swap signal; shared industry mule-account list.	API lookup; subject to contracts and consent.

Data quality

- **Label delay:** a transaction is treated as "mature" only after 60 days; recent data is used for monitoring with partial labels, and training uses only mature windows (plus confirmed fraud from recent weeks).
- **Label noise:** some disputes are "friendly fraud" (customer lies); unreviewed transactions are assumed legitimate. Label sources are weighted by reliability and audited monthly on a sample.
- **Class imbalance:** ≈ 0.08% positives by count (≈ 0.25% by value). Handled with down-sampling of negatives and class weights; evaluation always on the original distribution.
- **Training–serving skew:** features are defined once in the feature store and computed by the same code for online and offline; a daily job compares online-logged features with recomputed offline features.
- **Validation gates:** schema, null rate, value ranges and distribution checks (Great Expectations) on every ingestion; a failed check stops the training pipeline.

- **Leakage control:** point-in-time joins only; features that are written *after* a decision (e.g. "account blocked") are excluded.

Privacy

- Processing is based on the legitimate purpose of fraud prevention stated in the terms of service and privacy notice, in line with Vietnam's Decree 13/2023/ND-CP on personal data protection.
- **Data minimisation:** no names, phone numbers or ID-card numbers in the feature store; user IDs are pseudonymised; card numbers are tokenised (PCI DSS scope stays outside the ML system); location is reduced to coarse cells.
- **No sensitive attributes** (gender, ethnicity, religion, health) are used as features; fairness is checked by comparing false-decline rates across regions, age bands and account tenure.
- Encryption in transit and at rest; access to raw data only for named roles and logged; retention 5 years for audit data, 24 months for model-training feature tables.

Data volume

Item	Estimate
Transactions	1.2 M / day → ≈ 440 M / year
All raw events (transactions, logins, device, decisions)	≈ 15 M events / day, ≈ 25 GB / day compressed → ≈ 9 TB / year
Positive labels	≈ 1,000 / day → ≈ 350,000 / year
Training set (12 months, negatives down-sampled 1:50)	≈ 18 M rows × ≈ 120 features ≈ 10 GB Parquet
Online feature store	≈ 6 M active users + 45 k merchants + 8 M devices → ≈ 30 GB in Redis

3. Goals and Metrics

3.1 Goal Hierarchy

Goals are organised top-down: each model goal exists because it supports a system goal, and each system goal because it supports a business goal. This keeps the data-science work tied to money and customer experience rather than to an abstract accuracy number.

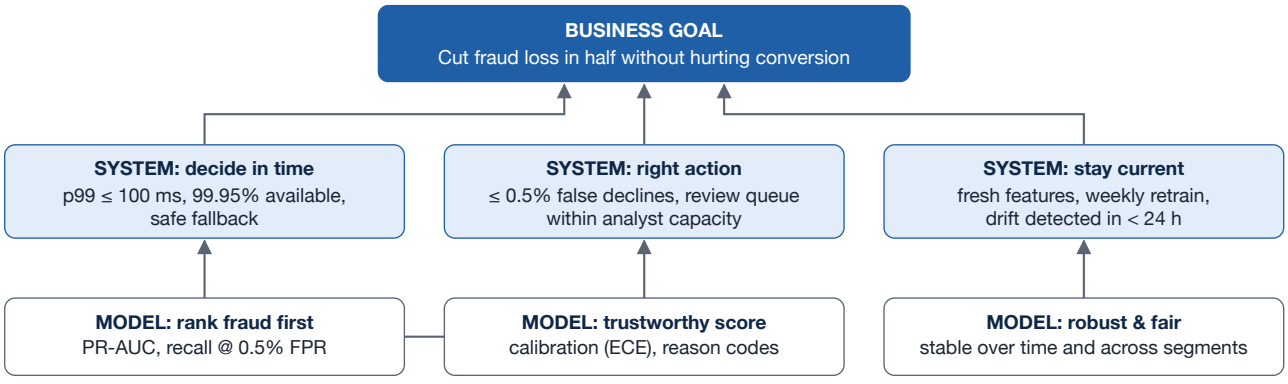


Figure 1 – Goal hierarchy: model goals serve system goals, which serve the business goal.

3.2 Business Goals

#	Goal	Metric (how it is measured)
B1	Reduce money lost to fraud	**Fraud loss rate** = refunded fraud value ÷ GMV, monthly, measured on matured (60-day) cohorts.
B2	Protect conversion and customer experience	**False decline rate** = good payments blocked or abandoned at challenge ÷ all payments; **challenge rate**; complaints about blocked payments per 10,000 users.
B3	Lower fraud-operations cost	**Review cases per day** and **review precision** (share of reviewed cases confirmed as fraud); median hold time.
B4	React faster to new fraud patterns	Days from first detected new pattern to a deployed model or rule covering it.

3.3 System Goals

#	Goal	Metric
S1	Decide within the payment budget	p50 / p99 scoring latency; timeout rate.
S2	Be available and degrade safely	Monthly availability; share of decisions using the fallback; error rate.
S3	Use fresh data	Streaming feature lag (event → online store), p95.
S4	Keep the model current and safe to change	Retraining pipeline success rate; time to roll back; drift alerts acknowledged in < 24 h.
S5	Explain and audit decisions	Share of non-ALLOW decisions with reason codes and complete audit record (target 100%).

3.4 Model Goals

#	Goal	Metric
M1	Rank fraudulent payments above legitimate ones	**PR-AUC** on a time-based hold-out month (primary offline metric); ROC-AUC as secondary.
M2	Catch fraud at an acceptable interruption rate	**Recall at 0.5% FPR** (count-based) and **value-weighted recall** at the same operating point.
M3	Produce a calibrated probability for the cost policy	Expected calibration error (ECE) and Brier score.
M4	Stay robust and fair	Recall and FPR per segment (new vs. old accounts, transaction type, region); score PSI week-over-week.

3.5 Baselines and Acceptance Thresholds

The **baseline** is the current rule engine, replayed on the same hold-out month. A simple logistic-regression model is the second baseline, to prove that a more complex model is worth its cost. A new model is promoted only if it meets every "minimum to launch" value offline *and* during a 2-week shadow run.

Metric	Baseline (rules today)	Minimum to launch	Target (6 months)	Level

Fraud loss rate (bps of GMV)	6.0	≤ 4.5	≤ 3.0	Business
False decline rate	1.2%	≤ 0.8%	≤ 0.5%	Business
Review cases / day · precision	9,000 · 4%	≤ 5,000 · ≥ 8%	≤ 3,000 · ≥ 15%	Business
Scoring latency p99	n/a (rules ≈ 20 ms)	≤ 120 ms	≤ 100 ms	System
Availability (monthly)	99.9%	99.9%	99.95%	System
Streaming feature lag p95	n/a	≤ 30 s	≤ 5 s	System
PR-AUC (hold-out month)	0.22 (rules) / 0.38 (log. reg.)	≥ 0.50	≥ 0.60	Model
Recall @ 0.5% FPR (count)	40%	≥ 65%	≥ 75%	Model
Value-weighted recall @ 0.5% FPR	45%	≥ 70%	≥ 80%	Model
Calibration ECE	n/a	≤ 0.02	≤ 0.01	Model
Worst-segment FPR ÷ overall FPR	2.5×	≤ 2.0×	≤ 1.5×	Model (fairness)
Score PSI week-over-week	–	< 0.1 normal · 0.1–0.25 investigate · > 0.25 retrain		Model (drift)

> **Operating point.** Thresholds are not fixed by hand. For each action the policy chooses the threshold that minimises expected cost: *cost* = *P(fraud)* × *amount* × *loss-share* – *(1 – P(fraud))* × *friction cost*, where friction cost is the estimated revenue and churn cost of interrupting a good customer (≈ 2% abandonment for a challenge, ≈ 15% for a block). The cut-offs are then capped so that the review queue stays within analyst capacity.

4. High-level Architecture Design

4.1 Architecture Overview

The system has three planes that share one feature definition layer: an **online plane** that scores each payment synchronously, a **streaming plane** that keeps behavioural features fresh, and an **offline plane** that builds training data, trains, evaluates and releases models. Monitoring and the analyst feedback loop connect the planes.

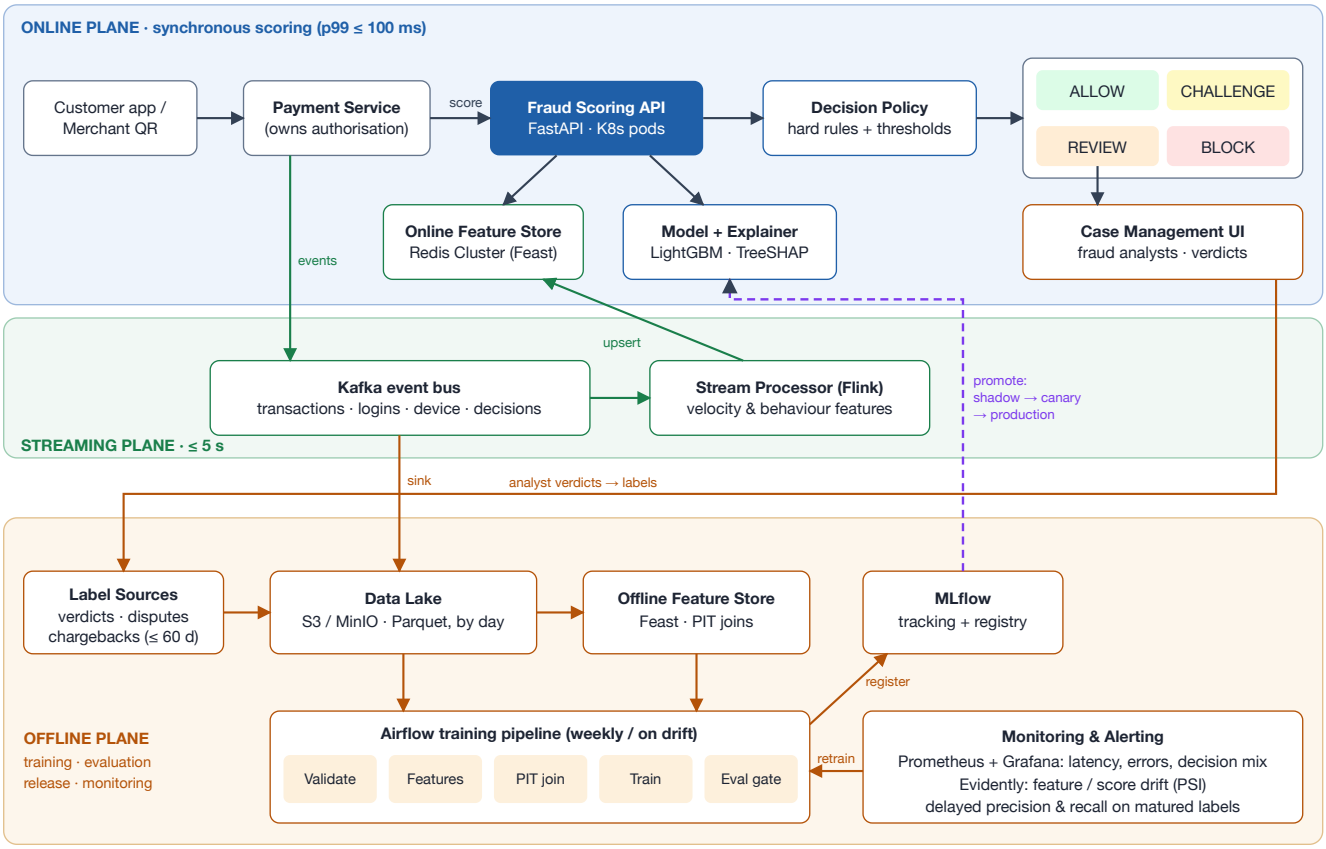


Figure 2 – High-level architecture. Solid arrows are data flows; the dashed arrow is model release.

4.2 Data Flow

A. Online scoring path (one payment)

1. The customer confirms a payment in the app; the **Payment Service** validates balance and limits and calls the **Fraud Scoring API** with the transaction payload (timeout 150 ms).
2. The API fetches ≈ 120 pre-computed features for the user, device, recipient and merchant from the **online feature store** in one batched Redis call (≈ 5 ms) and adds request-time features (amount ÷ user's 30-day median, hour of day, is-new-recipient).
3. The **LightGBM model** (loaded in-process) returns a calibrated probability (≈ 3 ms); **TreeSHAP** computes the top contributing features, which are mapped to reason codes.
4. The **Decision Policy** first applies hard rules, then compares the score with amount-dependent thresholds and returns ALLOW / CHALLENGE / REVIEW / BLOCK with reason codes.
5. The Payment Service executes the action (calls the Authentication Service for CHALLENGE, holds funds for REVIEW) and publishes the payment event and the decision record to **Kafka**.

B. Streaming feature path

6. **Flink** consumes payment, login and device events, keyed by user, device and recipient, and maintains sliding-window aggregates (count and sum of transfers in 10 min / 1 h / 24 h, distinct recipients, failed-OTP count, new-device flag).
7. Updated aggregates are upserted into Redis within seconds, so the *next* payment of a fast-moving fraudster already sees them.

C. Offline learning and feedback path

8. A Kafka sink writes all events and decision logs to the **data lake**, partitioned by day. Labels from analyst verdicts, disputes and partner-bank chargeback files are loaded daily.
9. **Airflow** runs the training pipeline weekly: validate → build features → point-in-time label join → train → evaluate. The candidate model and its metrics are logged to **MLflow** and registered.
10. A candidate that passes the evaluation gate is promoted through **shadow** → **canary** → **production**. Monitoring compares live behaviour with the training baseline and can trigger an early retrain.

4.3 ML Pipeline Stages

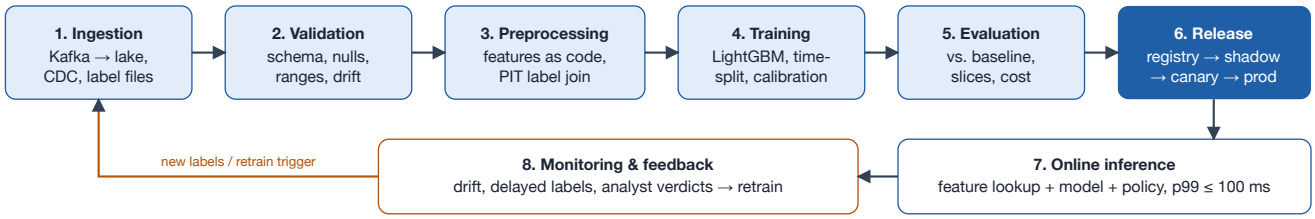


Figure 3 – ML pipeline stages and the feedback loop.

Stage	Key decisions
1. Ingestion	Events from Kafka are written to Parquet by day; core-database tables are copied by change-data-capture; partner-bank chargeback files are parsed and matched to `transaction_id`.
2. Validation	Great Expectations suites per table; checks for schema changes, null rates, value ranges, duplicate IDs, sudden volume changes. Failure stops the run and alerts the on-call engineer.
3. Preprocessing & features	Feature groups: *transaction* (amount, type, hour, amount z-score), *velocity* (counts and sums in 10 min–30 d), *device & session* (device age, accounts per device, failed OTPs), *counter-party* (recipient account age, inbound fan-in, merchant dispute rate), *account* (tenure, KYC level, linked sources). Point-in-time join of labels; label window 60 days.
4. Training	LightGBM binary classifier; training on months M–13 … M–2, validation on M–1 (mature labels only); negatives down-sampled 1:50 with weights restored; hyper-parameters tuned with Optuna; isotonic calibration on the validation month.
5. Evaluation	Compared with the current production model, the rule engine and logistic regression on the same month: PR-AUC, recall at 0.5% FPR, value-weighted recall, ECE, per-segment metrics, simulated business cost. Gate: all "minimum to launch" thresholds and no segment worse than production by more than 10%.
6. Release	Registered in MLflow with data snapshot ID and git commit. **Shadow** 2 weeks (scores logged, not used) → **canary** 10% of traffic for 1 week → 100%. Automatic rollback if latency, error rate or decision mix leaves its band.

7. Inference	Model artefact baked into the scoring image and loaded in memory; same feature code as training through the feature store.
8. Monitoring	Real-time: latency, errors, decision mix, fallback rate. Daily: feature and score PSI, online/offline feature parity. Weekly: precision/recall on newly matured labels; analyst override rate.

4.4 Component Description

Component	Purpose and responsibility	Technologies considered	Choice and reason
Fraud Scoring API	Receive request, gather features, call model and policy, return decision, emit log.	FastAPI, gRPC (Go), BentoML, Seldon	**FastAPI + Uvicorn** on Kubernetes: team skill in Python, model in-process (no extra hop), meets latency with LightGBM.
Model	Estimate fraud probability from tabular features.	Logistic regression, XGBoost, LightGBM, deep sequence models	**LightGBM**: best accuracy on tabular data for the latency budget; native missing-value handling; TreeSHAP explanations.
Decision Policy	Hard rules + cost-based thresholds → action; versioned config editable by risk managers.	Drools, custom Python, OPA	**Python module with YAML config** stored in git, four-eyes approval; simple and auditable.
Online Feature Store	Low-latency read of latest feature values.	Redis, DynamoDB, Cassandra	**Redis Cluster via Feast**: sub-ms reads, team already runs Redis; Feast keeps online and offline definitions identical.
Event bus	Durable, ordered event stream for features, logs and the lake.	Kafka, Pulsar, Kinesis	**Kafka**: already used by the Payment Service.
Stream Processor	Sliding-window aggregates with exactly-once state.	Flink, Spark Structured Streaming, Kafka Streams	**Flink**: event-time windows with low latency; Kafka Streams as a lighter alternative.
Data Lake & Offline Store	Historical events, labels, training snapshots.	S3/MinIO + Parquet, Delta Lake, BigQuery	**S3-compatible storage + Parquet** (Delta Lake later for time travel); Feast for point-in-time joins.
Orchestration	Schedule and track ingestion, validation, training, evaluation.	Airflow, Kubeflow Pipelines, Prefect	**Airflow**: mature, simple retries and back-fills; team has experience.
Experiment tracking & registry	Record runs, metrics, artefacts; manage model stages.	MLflow, Weights & Biases, SageMaker Registry	**MLflow**: open source, self-hosted (data stays in-country), model stages.

Data validation	Stop bad data before training.	Great Expectations, TFDV, Pandera	**Great Expectations**: readable suites, HTML reports for audits.
Monitoring	System health, drift, delayed model quality.	Prometheus/Grafana, Evidently, Datadog	**Prometheus + Grafana** for service metrics; **Evidently** jobs for drift reports; alerts to on-call.
Case Management UI	Analyst review, verdicts, overrides.	Build in-house, vendor tool	Extend the existing back-office tool with score, reasons and a verdict button that writes labels.
CI/CD	Test and ship code, pipelines and models.	GitHub Actions, GitLab CI, Argo CD	**GitHub Actions** for tests and image builds; **Argo CD** for deployment to Kubernetes.

5. Trade-off Analysis

Five significant trade-offs shaped the design. For each, the options, the analysis and the decision are given.

5.1 Accuracy vs. Latency

Options. (a) Logistic regression – < 1 ms, PR-AUC ≈ 0.38; (b) LightGBM with ≈ 500 trees – ≈ 3 ms, PR-AUC ≈ 0.58 (expected); (c) a sequence model (Transformer over the last 100 events) or a graph neural network over the transfer graph – possibly +0.03–0.05 PR-AUC but 30–80 ms on GPU plus heavier feature retrieval.

Analysis. The fraud check has at most 100 ms of the payment's 300 ms budget, and feature retrieval plus network already use ≈ 20–30 ms. A deep model would consume most of the remaining budget at p99, need GPU serving and make explanations harder. Most of the signal that a sequence or graph model would learn can be approximated with engineered velocity and counter-party features computed by Flink.

Decision. LightGBM with engineered streaming features. Graph signals (e.g. "recipient is two hops from a known mule") are pre-computed offline every hour and served as features, moving the expensive part out of the request path. A deep model may be revisited as an asynchronous second-stage scorer for REVIEW cases only.

5.2 Freshness vs. Cost

Options. (a) Daily batch features only; (b) streaming features for fast-changing signals plus batch for slow ones; (c) everything streaming plus continuous (online) model training.

Analysis. ATO attacks empty an account in minutes: a "transfers in the last 10 minutes" feature that is a day old is useless. On the other hand, merchant dispute rates or account tenure change slowly and do not justify a streaming job. Continuous training is expensive, hard to validate, and can be poisoned by fraudsters; labels arrive with days or weeks of delay anyway, so the model weights cannot be truly "real-time".

Decision. Hybrid (b): ≈ 25 velocity and session features via Flink (≤ 5 s lag), the rest computed daily; model retrained **weekly**, not continuously, with emergency retraining on drift. This gives most of the freshness benefit for roughly a third of the cost of an all-streaming design.

5.3 Simplicity vs. Performance

Options. (a) One global model; (b) separate models per transaction type (P2P, QR, top-up, withdrawal); (c) a global model plus a stacked ensemble.

Analysis. Per-type models could fit each fraud pattern better, but they multiply training pipelines, monitoring dashboards, thresholds and on-call complexity by four, and the smaller types would have too few positive labels. The assignment's guidance and the team size (3 data scientists, 2 ML engineers) both favour one model.

Decision. One global LightGBM with transaction type as a feature and **per-type thresholds** in the decision policy. Segment metrics are monitored; a separate model would be introduced only if one segment stays below its target for two consecutive months.

5.4 Automation vs. Control

Options. (a) The model decides everything automatically; (b) every flagged payment is reviewed by a human; (c) graduated automation with human oversight.

Analysis. Full automation is fast but can block many good customers when the model misbehaves (e.g. after a data bug), and regulators expect human accountability for significant decisions. Full human review is impossible at 1.2 M payments per day and adds hours of delay.

Decision. Graduated automation (c): low risk → ALLOW automatically; medium → CHALLENGE (the customer, not an analyst, resolves it); high-value medium-high → REVIEW by an analyst; very high → BLOCK, with an easy appeal through customer support. Humans control the *policy* (thresholds, hard rules) through a reviewed config, model releases need an approval in MLflow, and automatic guards roll back a model if the decision mix moves outside its normal band.

5.5 Privacy vs. Personalisation

Options. (a) Use every available signal, including contacts, precise GPS and installed apps; (b) use only transaction-related and coarse device data; (c) (b) plus privacy-preserving extras with consent.

Analysis. Rich personal data would improve the model a little, but it conflicts with data-minimisation under Decree 13/2023/ND-CP, increases the impact of a breach, and erodes customer trust. Experience in fraud detection shows that most of the predictive power comes from behaviour (velocity, device, counter-party), not from personal attributes.

Decision. (b) with pseudonymised IDs, coarse location cells and no sensitive attributes. Optional signals (telco SIM-swap check) are used only with consent and a contract. Fairness metrics per segment are reported monthly to Compliance.

5.6 Summary

Trade-off	Chosen side	What we give up · how it is mitigated

Accuracy vs. latency	LightGBM, ≈ 3 ms	Some accuracy of deep models · graph features pre-computed offline; second-stage model possible for REVIEW.
Freshness vs. cost	Hybrid streaming + batch; weekly retrain	Slow features up to 24 h old · only fast signals streamed; drift-triggered retrain.
Simplicity vs. performance	One global model	Per-segment fit · per-type thresholds and segment monitoring.
Automation vs. control	Graduated automation	Some fraud passes as CHALLENGE · humans own policy; automatic rollback guards.
Privacy vs. personalisation	Behavioural data only	A little predictive power · consented external signals; fairness reporting.

6. Risks and Rollout Plan

Risk	Mitigation
Fraudsters adapt to the model	Weekly retraining; monitoring of new-pattern clusters; analysts can add emergency rules without a deployment.
Feedback loop bias (blocked payments never get labels)	Keep a small random "control" sample (0.5% of low-value flagged payments are challenged instead of blocked) to observe outcomes; use challenge results as weak labels.
Training–serving skew	Shared feature definitions in Feast; daily parity check; shadow period before every release.
Outage of Redis, Flink or the scorer	Timeouts, circuit breaker and fallback to the rule engine; multi-AZ deployment; runbooks and on-call.
Unfair impact on a customer segment	No sensitive features; monthly segment report; threshold caps per segment.

Rollout (≈ 6 months). Month 1–2: data pipelines, feature store, offline baseline. Month 3: first model, shadow mode next to the rule engine. Month 4: canary on 10% of P2P transfers, then all P2P. Month 5: all transaction types; analyst UI with reasons. Month 6: retire overlapping rules, review results against the targets in Section 3.5.

7. Conclusion

The proposed design replaces a saturated rule engine with a single, explainable LightGBM risk model fed by streaming and batch features, and a cost-aware decision policy that turns scores into graduated actions. It fits within the 100 ms latency budget, degrades safely to the existing rules, and keeps humans in control of policy and releases. The goal hierarchy ties every model metric to a system and business outcome: halving the fraud loss rate (6 → 3 bps of GMV), cutting false declines from 1.2% to 0.5% and shrinking the review queue by two thirds. The chosen trade-offs favour simplicity, freshness where it matters, and privacy, leaving clear upgrade paths (graph features, a second-stage model) once the base system has proven its value in production.